

Data: 30 de Abril de 2026
Projeto: BarberPro - Sistema Web para Barbearia
Contexto: Projeto Integrador da Univesp
Versao: 2.0.0

1. O Que E o BarberPro?

O BarberPro e um sistema web para gestao de barbearias, desenvolvido como parte do Projeto Integrador da Univesp. E um painel administrativo completo onde o dono de uma barbearia pode:

- Autenticar-se de forma segura (login com email e senha)
- Cadastrar e gerenciar barbeiros (nome, telefone, especialidades, comissao, status)
- Cadastrar e gerenciar servicos (corte, barba, combo, com precos e duracao)
- Gerenciar clientes com consentimento LGPD
- Criar e gerenciar agendamentos com integracao financeira
- Ver um dashboard com indicadores operacionais
- Receber pagamentos via PIX (integracao preparada)
- Gerar relatorios diarios com receitas, agendamentos e movimentacoes

Como Funciona por Dentro?

Frontend: Next.js 14 com App Router, interface dark mode responsiva, Tailwind CSS, componentes shadcn/ui.

Backend e Banco de Dados: Supabase com autenticacao via cookies HttpOnly, PostgreSQL com Row Level Security (RLS), integrado via Server Actions.

Validacao e Seguranca: Zod no client e server, webhook PIX com verificacao criptografica timing-safe.

Multi-Tenant: Cada dado tem barbearia_id isolando informacoes por barbearia.

2. Stack Tecnologica

Tecnologia	Versao	Uso
Next.js	14.2.35	Framework web (App Router)
React	18.3.1	Biblioteca de UI
TypeScript	5.8.3	Tipagem estatica (strict)
Supabase	N/A	Autenticacao, banco PostgreSQL, storage
Tailwind CSS	3.4.17	Estilizacao (dark mode)
shadcn/ui	N/A	Componentes UI reutilizaveis
Lucide React	0.513.0	Icones
Zod	4.0.5	Validacao de formularios
PostCSS	8.4.31	Transformacao CSS
ESLint	8.56.0	Linting de codigo

3. Estrutura de Pastas

```

barberpro/
├── Design System para BarberPro/      # Projeto Next.js
│   ├── app/
│   │   ├── (dashboard)/
│   │   │   ├── layout.tsx            # Layout principal com sidebar
│   │   │   ├── dashboard/page.tsx    # Dashboard principal
│   │   │   ├── dashboard/barbeiros/  # CRUD de barbeiros
│   │   │   ├── dashboard/servicos/   # CRUD de servicos
│   │   │   ├── dashboard/relatorios/ # Relatorio diario (nova)
│   │   │   ├── clientes/             # CRUD de clientes
│   │   │   ├── agendamentos/         # CRUD de agendamentos
│   │   │   └── components/           # Componentes compartilhados (FormWithReload)
│   │   ├── login/                   # Pagina de login
│   │   └── components/              # Componentes de UI (Card, AppointmentCard)
│   ├── lib/supabase/                # Cliente Supabase (SSR, middleware)
│   ├── supabase/migrations/         # Migrations do banco de dados
│   └── middleware.ts                 # Middleware de autenticacao
├── relatorio-status.md               # Este arquivo
├── relatorio-status.pdf              # Versao PDF deste arquivo
└── README.md                        # Documentacao do projeto

```

4. Funcionalidades Implementadas

4.1 Autenticacao

- Login com email e senha via Supabase Auth
- Sessoes persistentes com cookies HttpOnly
- Middleware protege rotas privadas (/dashboard, /clientes, /agendamentos)
- Redirecionamento automatico para /login se nao autenticado
- Bootstrap automatico de perfil admin

4.2 CRUD de Barbeiros

- Criar, editar e excluir barbeiros
- Campos: nome, CPF, telefone, email, especialidades, comissao, valor minimo
- Status ativo/inativo
- Link para ver receitas de cada barbeiro
- **Soft delete** → **Hard delete**: Exclusao real com `ON DELETE SET NULL` nas FKs
- CPF/email reutilizaveis apos exclusao (indice parcial)

4.3 CRUD de Servicos

- Criar, editar e excluir servicos
- Vinculo com categorias (Cortes, Barba, Combo, etc.)
- Campos: nome, descricao, duracao, preco, preco promocional, buffer

4.4 CRUD de Clientes

- Cadastro completo com consentimento LGPD (promocoos WhatsApp, historico)
- Campos: nome, telefone, email, CPF, data nascimento, observacoes, preferencias
- Vinculo com barbeiro que cadastrou
- Sistema de indicacao entre clientes

4.5 Agendamentos

- CRUD completo integrado com transacoes financeiras
- Campos: cliente, barbeiro, servico, data inicio/fim, status, valor, gorjeta, forma de pagamento
- Status: pendente, confirmado, em andamento, concluido, cancelado, no show
- Formas de pagamento: PIX, cartao credito/debito, dinheiro, fiado
- Criacao automatica de transacao financeira ao criar agendamento

4.6 Dashboard

- KPIs: Agendamentos hoje, Faturamento hoje, Receita acumulada, Barbeiros ativos
- Proximos agendamentos (5 mais recentes)
- Estatisticas rapidas (total de agendamentos, receitas, barbeiros, servicos)

4.7 Receitas por Barbeiro

- Pagina dedicada `/dashboard/barbeiros/[id]/receitas`
- Receita total, pendente e comissao estimada
- Transacoes agrupadas por dia com detalhes

4.8 Relatorio Diario (NOVO)

- Rota: `/dashboard/relatorios`
- Filtro de data com recarregamento forçado
- Resumo do dia:
 - Receita total (pago) e pendente
 - Barbeiros novos cadastrados
 - Total de registros adicionados/excluidos
- **Receitas por Barbeiro:** breakdown com valores pagos/pendentes por barbeiro
- **Agendamentos:** lista completa com cliente, servico, barbeiro, horario, status, pagamento, gorjeta
- **Exportar PDF:** botao que usa `window.print()` para gerar PDF via navegador

5. Banco de Dados

Tabelas Principais

Tabela	Descricao
perfis	Perfis de usuario vinculados ao Supabase Auth
barbeiros	Cadastro de barbeiros com comissao e especialidades
clientes	Cadastro de clientes com consentimento LGPD
categorias_servicos	Categorias para organizacao de servicos
servicos	Servicos oferecidos pela barbearia
agendamentos	Agendamentos com integracao financeira
transacoes	Transacoes financeiras (receitas, despesas)
notificacoes	Sistema de notificacoes
audit_log	Log de auditoria para compliance
arquivos	Gestao de arquivos/uploads

Relacionamentos de Integridade (FKs)

Tabela	Coluna FK	Referencia	ON DELETE
barbeiros	perfil_id	perfis(id)	SET NULL
clientes	criado_por	barbeiros(id)	SET NULL
agendamentos	barbeiro_id	barbeiros(id)	SET NULL
agendamentos	cliente_id	clientes(id)	SET NULL
agendamentos	servico_id	servicos(id)	SET NULL
transacoes	agendamento_id	agendamentos(id)	SET NULL

Nota importante: Todas as FKs de agendamentos e transacoes usam `ON DELETE SET NULL`. Isso garante que ao excluir um barbeiro, cliente ou agendamento, os dados historicos de receitas e relatorios **nao sao perdidos**. O campo FK fica NULL e a interface exibe "(Barbeiro excluido)" ou similar.

Row Level Security (RLS)

- Todas as tabelas tem RLS habilitado
- Politicas de leitura para admin e barbeiros
- Politicas de escrita para admin/atendente
- Agendamentos com `barbeiro_id IS NULL` sao visiveis ao admin

6. Correcoes e Melhorias Aplicadas

Correcao 1: Estrutura de Server Actions

Problema: Server Actions dentro de arquivos de pagina causavam erros de compilacao ("use server" em escopo incorreto).

Solucao: Criados arquivos `actions.ts` separados para cada modulo (barbeiros, clientes, agendamentos).

Correcao 2: Dados stale e duplicacao no CRUD

Problema: Ao criar/editar/excluir registros, os dados não atualizavam corretamente na tela, duplicando registros ou mostrando itens já excluídos.

Solução: Criado componente `client FormWithReload` que intercepta o submit, executa a action e recarrega a página com `window.location.reload()`. Mesmo padrão aplicado ao `DeleteButton`.

Correção 3: Feedback visual em operações

Problema: Botões não davam indicativo de que a operação estava em andamento.

Solução: Adicionado estado `pending` com desabilitação do botão e textos "Salvando..." / "Excluindo..." para evitar duplo clique.

Correção 4: Dashboard sem receita acumulada

Problema: Dashboard mostrava apenas receita do dia, sem total acumulado.

Solução: Adicionado KPI "Receita Acumulada" com query de todas as transações pagas.

Correção 5: Preservação de dados históricos ao excluir

Problema: Ao excluir um barbeiro, todos os agendamentos e receitas dele eram apagados (CASCADE), impossibilitando relatórios históricos.

Solução:

- Migrations para mudar todas as FKs relevantes de `ON DELETE CASCADE` para `ON DELETE SET NULL`
- CPF/email agora usam índices parciais (validados apenas para registros ativos), permitindo reutilização
- Interface mostra "(Barbeiro excluído)" quando o barbeiro foi removido
- RLS atualizada para permitir leitura de agendamentos com `barbeiro_id IS NULL`

Correção 6: Relatório diário não atualizava ao filtrar data

Problema: O botão de filtrar data no relatório não recarregava os dados.

Solução: Componente `client DateFilter` usa `window.location.href` para forçar recarregamento completo da página com o novo parâmetro de data.

Correção 7: Timezone incorreto no filtro de data

Problema: `new Date("2026-04-30")` criava data com offset de timezone, causando busca no dia errado no banco (que armazena em UTC).

Solução: Usado `Date.UTC()` para construir datas em UTC corretamente, alinhando com o formato de armazenamento do Supabase.

Nova Funcionalidade 1: Receitas por Barbeiro

Rota: `/dashboard/barbeiros/[id]/receitas`

- Página dedicada com receitas agrupadas por dia
- Cards com receita total (pago), receita pendente e comissão estimada
- Lista detalhada por dia com cliente, serviço, horário, forma de pagamento e status
- Link "Ver Receitas" adicionado em cada card de barbeiro

Nova Funcionalidade 2: Relatório Diário

Rota: `/dashboard/relatorios`

- Filtro de data com recarregamento forçado
- Resumo: receita total, receita pendente, barbeiros novos, total adicionados/excluídos
- Receitas por barbeiro (com valores pagos e pendentes)
- Lista completa de agendamentos do dia selecionado
- Botão "Exportar PDF" usa `window.print()` para gerar PDF via navegador

7. Como Rodar o Projeto

1. Instale as dependências:

```
npm install
```

2. Copie as variáveis de ambiente:

```
copy .env.example .env.local
```

3. Preencha no `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

4. Rode as migrations no Supabase SQL Editor (em ordem):

- supabase/migrations/20260429_schema_final_rls_lgpd.sql (schema principal)
- supabase/migrations/20260430_preserve_on_delete_v2.sql (FKs SET NULL)
- supabase/migrations/20260430_unique_active_only.sql (indices parciais CPF/email)
- supabase/migrations/01_fix_rls_null_barbeiro.sql (RLS para barbeiro_id NULL)

5. Rode o projeto:

```
npm run dev
```

6. Build de producao:

```
npm run build  
npm run start
```

8. Historico de Commits

Versao 2.0.0 (30/04/2026)

1. **fix**(crud): adicionar FormWithReload **com** refresh automatico apos submit
2. **feat**(ui): adicionar feedback visual de loading em botoes de CRUD
3. **feat**(relatorios): preservar dados ao excluir barbeiros/clientes e adicionar relatorio diario
4. **fix**(relatorios): filtrar ativos, corrigir formulario de **data** e migration v2
5. **fix**(relatorios): corrigir contagem de barbeiros, receita por agendamento e constraints de unicidade
6. **fix**(relatorios): corrigir filtro de **data** e FK de agendamentos
7. **fix**(relatorios): usar **window.location.href** para forcar refresh ao filtrar **data**
8. **fix**(relatorios): hard **delete** **com** SET NULL, UTC date filter e vinculos perdidos
9. **feat**(relatorios): relatorio diario **com** export PDF, correcao de **data** e contagens

Versao 1.x (29/04/2026)

1. **feat**(next): migrar estrutura vite para next14 app router
2. **feat**(auth): implementar supabase SSR com middleware e server actions
3. **feat**(crud): adicionar crud de barbeiros e servicos com zod
4. **feat**(clientes): implementar crud completo com consentimento lgpd
5. **feat**(agendamentos): implementar crud integrado com transacoes financeiras
6. **fix**(agendamentos): mover syncTransacao para escopo do modulo
7. **fix**(auth): bootstrap automatico de perfil admin para RLS
8. **chore**(db): criar migrations com barbearia_id e rls
9. **docs**(readme): documentar setup, env e checklist univesp

Relatorio gerado em 30/04/2026